

Robot Mapping with LiDAR Data

Omar Ahmed Geasa
Mechatronics Engineering
Tanta University
Omargeasaa@gmail.com

Abstract:

This mini-project demonstrates how to manipulate raw LiDAR data using point clouds to construct a visual map of the environment. The work focuses on applying affine transformations concepts from linear algebra to convert data from the robot's local coordinate frame into a world reference frame. This project shows how affine transformations enable accurate alignment of multiple LiDAR scans for real-time robotic mapping. The dataset used in this work was collected by the Cassie bipedal robot at the University of Michigan.

I- Introduction:

Many autonomous systems rely on perception sensors such as stereo cameras and LiDAR to localize, map, and navigate in unknown environments. By using these sensors, the robot can construct a spatial representation of its surroundings and how it interacts with them. LiDAR systems generate dense point clouds that capture the 3D structure of the environment with high precision which allows us to visualize how the robot operates within its surroundings.

However, LiDAR collects data in its own local coordinate frame, which does not always align with the robot's reference frame. As the robot moves, both the LiDAR and robot coordinate frames change with respect to the environment, causing misalignment among successive scans. Therefore, it is necessary to transform all robot data into a common world or global coordinate frame in order to construct the map. This can be achieved by computing the affine transformation between the local and global coordinates as the robot moves through the environment.

This project affine focuses on implementing transformations using linear algebra to transform LiDAR point cloud data. The project demonstrates how local sensor data can be transformed into a unified global map representation suitable for robotic mapping applications in addition to how large-scale data can be efficiently processed using matrix operations, which is

achieved by using real dataset expressing translation, and rotation operations in homogeneous coordinates.

II- Background:

i- Affine Transformation:

An affine transformation is a geometric mapping that preserves straight lines and parallelism between points, meaning that sets of parallel lines remain parallel after the transformation. It combines multiple linear operations, such as translation, rotation, scaling, and shearing, into a unified mathematical framework, although it does not necessarily preserve angles or distances.

In computer graphics, affine transformations enable the rendering of dynamic scenes and the manipulation of 3D models. In computer vision, they are applied to correct image distortions caused by wide-angle lenses or changes in camera perspective. By representing these transformations through homogeneous coordinates, complex sequences of operations can be expressed as a single matrix multiplication

Mathematically, if A is a linear transformation matrix and t is a translation vector, the affine transformation of a point x can be expressed as:

$$f(x) = y = Ax + t$$

ii- Homogeneous Coordinates

While affine transformations describe how points are mapped between coordinate frames through rotation, scaling, and translation, the translation component cannot be represented purely as a matrix multiplication in standard Cartesian coordinates. Homogeneous coordinates are introduced to express all transformation operations in a single matrix.

In homogeneous coordinates, a 2D point $[x, y]^T$ becomes $[x, y, 1]^T$, and a 3D point $[x, y, z]^T$ becomes $[x, y, z, 1]^T$. This allows the affine

transformation to be written as a single matrix–vector multiplication:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} A & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Here, A represents the linear part of the transformation (rotation, scaling, or shear), and t is the translation vector.

The resulting matrix is known as the homogeneous transformation matrix, which is the matrix representation of an affine transformation expressed in homogeneous coordinates. This formulation makes it possible to combine multiple transformations into a single matrix multiplication. homogeneous coordinates. This formulation makes it possible to combine multiple transformations into a single matrix multiplication.

is used to convert point clouds from the LiDAR’s local frame to the global world frame, aligning sensor data with the robot’s estimated pose. Transformations form the mathematical foundation for describing how a robot interacts with the surrounding environment. Every component of a robotic system, such as the base, end-effector, LiDAR, or camera, has its own local coordinate frame, which changes dynamically as the robot moves. To interpret sensor measurements or control motion accurately, all these frames must be

related through well-defined geometric transformations.

In this project, the same principle is applied to LiDAR data collected by the robot. Each LiDAR scan is originally captured in the sensor’s local frame, which moves with the robot. By applying the homogeneous transformation matrix composed of the rotation matrix R and translation vector t , the data is transformed into the global frame. This step ensures that consecutive LiDAR scans are properly aligned, allowing a coherent 3D map of the environment to be constructed.

iii- Application in Robotics

In robotics, homogeneous transformation matrices are essential for describing how sensors and robot components move relative to one another. For example, in LiDAR-based mapping, a transformation matrix of the form

$$T = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

Where:

- $R_{3 \times 3}$ is a 3 x 3 rotation matrix
- $t_{3 \times 1}$ is a 3 x 1 translation vector
- $0_{1 \times 3}$ is a 1 x 3 row vector of zeros
- 1 is the number one

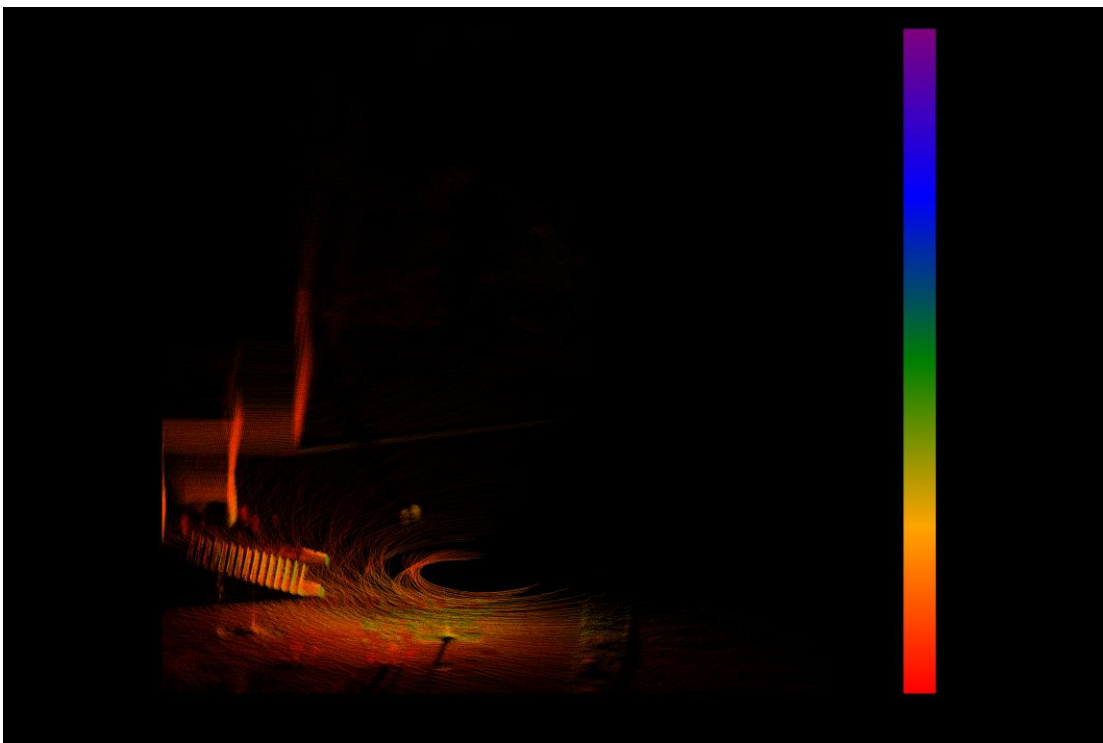


Figure 1. Motivation: Expected project output showing LiDAR-based 3D mapping

III- Methodology

This section outlines the methodology used to transform LiDAR data from the robot's local coordinate frame into the world coordinate frame. It is divided into three main parts:

- 1- 2D geometric transformations
- 2- 2D point cloud correction
- 3- 3D LiDAR mapping and visualization.

Each step builds upon the previous one to illustrate how affine transformations are used in robotic mapping.

Step 1: 2D Transformations and Homogeneous Coordinates

In this step, simple geometric shapes are plotted and transformed using matrix operations to illustrate the concept of affine transformations. These transformations combine translation, rotation, and scaling within a single mathematical framework. To represent these operations consistently, the coordinates are expressed in homogeneous form.

Before working with LiDAR data, it is important to understand the three core geometric operations used to manipulate point coordinates in 2D:

1.A: Translation

The first transformation applied to the square is translation which moves the points by a constant offset (t_x, t_y) . Each point of the square is shifted by 0.5 units along both the x-axis and y-axis to center the shape around the origin. Mathematically, this is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$$

The translated points are stored in an array named 'points2' for subsequent transformations.

Step 1.B: Translation Using Homogeneous Coordinates

To express translation as a matrix-vector multiplication, the points are represented in homogeneous coordinates. This allows translation, rotation, and scaling to share a unified mathematical form, simplifying computations on large datasets.

A 2D point $[x, y]^T$ is converted to its homogeneous representation by appending a constant 1, forming $[x, y, 1]^T$.

Translation is then expressed as:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Applying this transformation:

$$[x', y', 1]^T = [x + t_x, y + t_y, 1]^T$$

This unified form enables translation to be handled identically to other affine transformations.

Step 1.C: Rotation in Homogeneous Coordinates

Next, a rotation is applied using the 2D rotation matrix in counterclockwise direction by angle θ :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

For this step, the square is rotated by 45° ($\pi/4$ radians). The transformation matrix is constructed using $\theta = \pi/4$ and applied to all points.

Step 1.D: Scaling

Scaling adjusts the size of the square along the x- and y-axes by factors S_x and S_y . It is a non-rigid transformation that modifies the object's dimensions.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In this case, $S_x = 1.5$ and $S_y = 0.5$, stretching the square horizontally and compressing it vertically.

Step 1.E: Combining Transformations: Affine Transformation

All the transformations—translation, rotation, and scaling—can be expressed within a single affine framework. Instead of applying them sequentially, they can be multiplied together as homogeneous matrices:

$$T = \begin{bmatrix} 1.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}$$

This composite matrix represents the overall affine transformation and can be directly applied to the point set.



Figure 2. Demonstration of 2D geometric transformations applied to a square.

The original shape (green) undergoes **translation** (black), **rotation** (red), and **scaling** (blue).

The **final output** (blue) shows that performing all three steps sequentially yields the same result as applying a single

Step 2 – Working with 2D Point Clouds

This step introduces point cloud manipulation before moving to full 3D LiDAR data. It demonstrates how affine transformations can correct distortions in 2D datasets.

A point cloud is a set of points in space, each with geometric and sometimes intensity attributes. For a 2D point cloud, the dataset can be represented as a $3 \times N$ matrix where the first two rows contain x and y coordinates, and the third row holds intensity or color values normalized between 0 and 1.

$$\begin{bmatrix} x_1 & x_2 & \dots & x_N \\ y_1 & y_2 & \dots & y_N \\ I_1 & I_2 & \dots & I_N \end{bmatrix}$$

To correct the distortion, a known affine transformation is applied that maps the distorted coordinates to their true positions. The transformation matrix is given as:

$$T = \begin{bmatrix} -0.09239 & 0.038268 & 300 \\ -0.38268 & -0.923879 & 165 \\ 0 & 0 & 1 \end{bmatrix}$$

1. Load the data using `readdlm()` into a $3 \times N$ array.
2. Separate x - y coordinates and intensity values.
3. Convert each point to homogeneous coordinates $[x, y, 1]^T$.
4. Apply the affine transformation $P' = T \cdot P_h$
5. Plot the corrected points to visualize the recovered image.

After transformation, the scattered points re-align to form a readable image, demonstrating the usefulness of affine transformations in correcting spatial distortions.

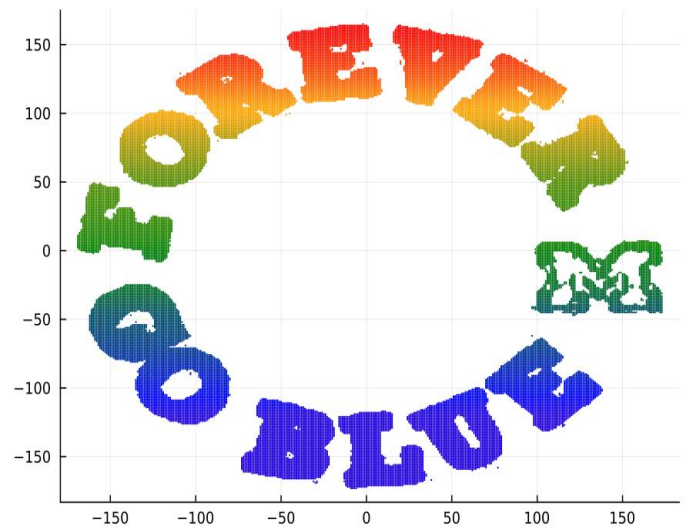
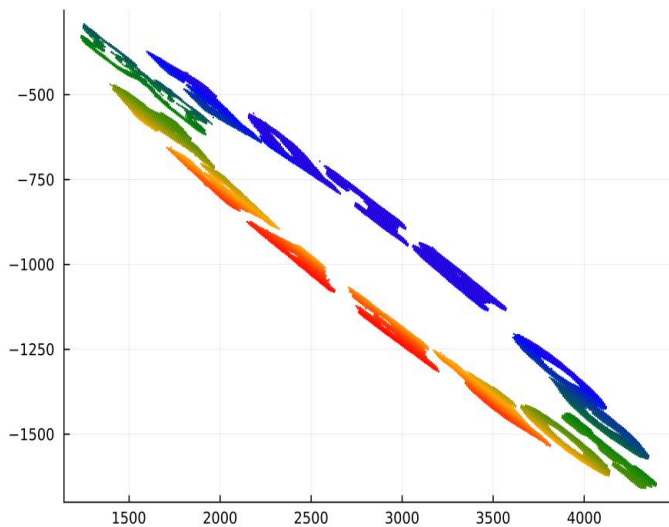


Figure 3. Comparison of 2D point clouds before and after applying affine transformations.

Step 3 – 3D LiDAR Mapping and Visualization

This step extends the previous concepts to 3D LiDAR data collected by the Cassie Blue robot. Affine transformations are used to register multiple LiDAR scans into a single coherent map.

Cassie's LiDAR captures point clouds at 10 Hz, each recording 3D coordinates, intensity values, and the robot's pose. Each scan is defined in the LiDAR sensor's local coordinate frame, which moves with the robot. Therefore, to combine scans, each must be transformed into a fixed world frame.

The `data_parser(id)` function loads and structures the data for each interval $id \in \{9, 10, 11, 12, 13\}$

it returns:

- `pointcloud_data`: $3 \times N$ array of $[x, y, z]$
- `Intensity_data`: vector of LiDAR intensities
- `R`: 3×3 rotation matrix
- `t`: 3×1 translation vector
- `pose`: Cassie's position and orientation at that frame

Each LiDAR scan is transformed to the world frame by the matrix:

$$T = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

and applied as $P' = T \cdot P_h$, where $P_h = [x, y, z, 1]^T$.

Part A – Single Frame

1. Parse data for $id = 13$.
2. Plot the robot's pose (position and orientation).
3. Overlay the raw LiDAR point cloud.

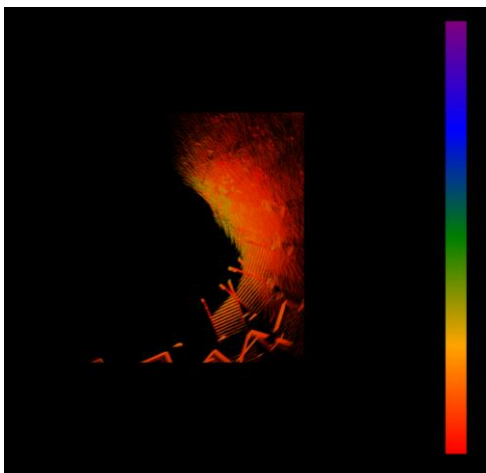


Figure 4. Raw LiDAR data collected before any processing or frame alignment.

4. Apply the transformation matrix T to convert the cloud into the world frame.
5. Re-plot the transformed cloud; the LiDAR rings should now be centered around the robot.

Part B – Building the Map (Fusing Scans)

To build a global map, multiple LiDAR scans from 9–13 s are fused together.

For each id :

1. Call `data_parser(id)`.
2. Build the homogeneous matrix $T_i = [R_i t_i, 0001]^T$
3. Transform the local point cloud: $P'_i = T_i \cdot P_{i,h}$.
4. Append P'_i to the global map array.

After looping over all frames, the result is a **dense, unified 3D point cloud** that represents the environment from multiple robot positions.

Each scan is transformed using its corresponding R and t matrices, compensating for the robot's motion. All transformed scans are merged into a single unified 3D point cloud representing the environment.

Animation

A GIF animation can be generated to visualize the mapping process in real time. Each frame adds a new LiDAR scan to the map, while a white dot marks the robot's position. The final animation illustrates Cassie moving and mapping simultaneously.

This concludes the LiDAR data transformation and mapping process. The resulting global map demonstrates how matrix-based affine transformations enable accurate 3D reconstruction of the environment from raw LiDAR data.

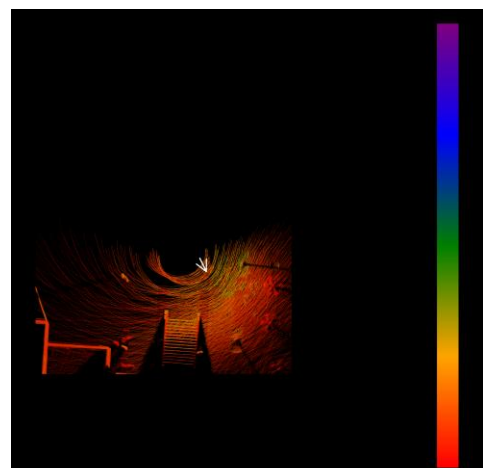


Figure 5. Corrected LiDAR point cloud for frame 9 after applying the homogeneous transformation.

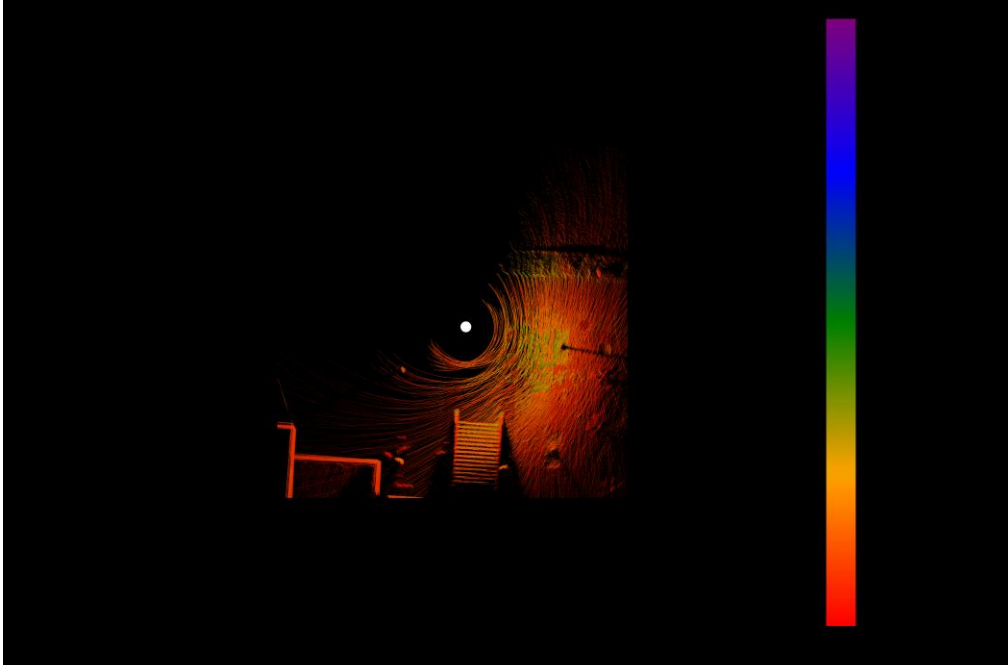


Figure 6. Animated visualization (GIF) showing Cassie's trajectory and accumulated LiDAR map after frame alignment.

IV- Results & Discussion:

After applying the affine transformation to the LiDAR point cloud data, the corrected results clearly demonstrate the effectiveness of the method. Initially, the raw LiDAR scans appeared misaligned due to the robot's continuous motion during data collection. Each scan was recorded in the LiDAR's local coordinate frame, resulting in overlapping and displaced point clouds when plotted together. This can be seen in *Figure 4*, where multiple scans produce duplicated and distorted representations of the same environment.

By constructing and applying the homogeneous transformation matrix

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix},$$

which combines rotation and translation into a transformation, the data from each frame were accurately projected into the world coordinate frame. The corrected results, shown in *Figure 5*, display concentric LiDAR rings centered about the robot's true position, with all scans properly aligned.

This confirms that the transformations successfully compensated for Cassie's motion and orientation changes between frames.

Finally, by sequentially fusing LiDAR data across five-time intervals (9–13 seconds), a global 3D map of the robot's surroundings was constructed.

This fusion process demonstrates how spatial information accumulates over time to produce a coherent and detailed representation of the environment.

An animated visualization (*Figure 6*) further illustrates Cassie's motion through the mapped space, with the white marker tracking the robot's estimated position in real time.

Assumptions and Limitations:

1. The experiment assumes that Cassie's true position and orientation at each frame are *known and accurate*. In real-world robotics, these values must typically be estimated using algorithms such as the Extended Kalman Filter (EKF) or Simultaneous Localization and Mapping (SLAM).
2. The transformation matrix (R, t) between the LiDAR and world frames is also assumed to be given and precise. In practical scenarios, determining these parameters involves sensor calibration and uncertainty handling, which are not covered in this project.

Despite these simplifications, the experiment effectively demonstrates the mathematical and computational principles required to perform large-scale spatial transformations which considered a foundation for more advanced techniques in robotic perception and localization.

V- Conclusion:

This project successfully demonstrated how affine and homogeneous transformations can be applied to real LiDAR data to reconstruct a 3D map from local sensor measurements.

Through stepwise implementation, beginning with 2D geometric transformations and extending to 3D LiDAR data, the study illustrated how linear algebra provides the mathematical backbone for robotic mapping and localization.

The results confirm that properly designed transformation matrices allow consistent alignment of LiDAR scans across multiple frames, enabling the robot to perceive and reconstruct its environment accurately. Although certain parameters such as Cassie's position and transformation matrices were provided, the process

mirrors real-world robotic perception pipelines that rely on continuous estimation and transformation of sensory data.

Future extensions could include integrating pose estimation algorithms to autonomously estimate the robot's position and orientation in real time, further bridging this demonstration with practical robotic applications.

Acknowledgement:

This project is based on materials from the [ROB 101: Computational Linear Algebra](#) course at the University of Michigan. Original concept and dataset by Prof. Jessy Grizzle, Prof. Maani Ghaffari, and Tribhi Kathuria. The Implementation, code, and documentation here are my own.